varp python API documentation

Instance functions

```
varpy.new(options)
```

Create a new varp instance from a dict of options

- {'size': size}
  Initial variable table size
- {'qtype': 'lifo'|'fifo'|'recursive'}
  use lifo/fifo strategy in bcp
- {'xref': x}
  use cross references if x is True
- {'hash': x}
  use hash table for clauses if x is True
- {'edge': x}
  use edge tables for 2-clauses if x is True

```
varpy.clone(vp, options)
```

Clone the varp instance using setting options from varpy.new with the
follow additions:

- {'level': l}
  clone bindings up until level **l**
- {'set': **clauseset** | [**clauseset**]}
  clone clauseset DELTA, GAMMA, BETA and/or ALPHA
- {'queue', x}
  clone bcp queue only if x is True.

where **clauseset** = 'delta'|'gamma'|'beta'|'alpha'

```
varpy.info(vp,  item)
```

Get varp information

- version
- 'bcp_counter'
- 'conflict_counter'
- 'max_conflicting'
- 'num_conflicting'
- 'number_of_variables'
- 'number_of_clauses'
- 'number_of_edges'
- 'number_of_dead_clauses'
- 'number_of_dead_edges'
- 'number_of_learnt_clauses'
- 'number_of_bound_variables'
- 'number_of_subst_variables'
- 'number_of_unbound_variables'

- 'clause_n_counter'
- 'clause_2_counter'
- 'clause_3_counter'
- 'clause_d_counter'
- 'edge_2_counter'
- 'edge_d_counter'
- 'size'
- 'qtype'
- 'max_level'
- 'min_level'
- 'max_bound'
- 'literal_size'
- 'literal_integer'
- 'value_packing'
- 'edge'
- 'xref'
- 'hash'
- 'init_phase'
- 'use_phase'

```
varpy.config(vp, item, value)
```

Set configurable items in varp

- 'max_conflicting'
  set max number of conflicts during bcp (<= MAX_CONFLICTING=1024)
- 'xref'
  turn on (True) or off (False) cross reference handling
- 'hash'
  turn on (True) or off (False) hash table handling
- 'qtype'
  set style of literal queueing in bcp
  where value is one of
  'lifo' | 'fifo' | 'recursive'

```
varpy.add_variable(vp [,is_atom])
```

Create a new variable. The variables is return as an index to the
next available variable in the variable table. Mark the new variable
as atom if is_atom is True. The atom status may later be queried with
variable info. is_atom defaults to True.

```
varpy.value(vp, x)
```

Return variable value as varp constant
varpy.**t** | varpy.**f** | varpy.**undefined**

```
varpy.bind(vp, x, [,l])
```

Bind variable x to True. If level **l** is given then that level
is used to the variable is bound on that level else the
variable is bound on the current level as set with varpy.set_level.

```
varpy.decide(vp, x [,l])
```

Bind variable x to True and mark x as a decision variable.
If level **l** is given then that level
is used to the variable is bound on that level else the
variable is bound on the current level as set with varpy.set_level.

```
varpy.subst(vp, x, y)
```

Substitute one literal for an other. Apply the substitution [**x**/**y**]
that is, all instances of **y** are replaced by **x** in all clauses.
The literal **y** is then linked to **x** so it will keep the
same status and value as that of **x**.
NOTE that cross references must be enabled before varpy.subst can be
used, that is a call to varpy.config(vp, 'xref', True) must have been made
prior a call to varpy.subst.

```
varpy.implication_clause(vp, x)
```

Return the clause index for the clause where literal **x** became a unit.

```
varpy.implication_level(vp, x)
```

Return the bind level for variable **x**, where it was assigned during bcp.

```
varpy.implication_pos(vp, x)
```

Return the position where literal **x** is found in the implication clause.

```
varpy.conflicting_clause(vp, i)
```

Return the i'th conflicting clause found during the last bcp. The number
of conflicting clauses that can be returned is 'num_conflicting'.

```
varpy.is_variable(vp, x)
```

Return **True** if literal **x** is unbound. Return **False** otherwise

```
varpy.is_bound(vp, x)
```

Return **True** if literal **x** is bound, through substition,
to an other variable. Return **False** otherwise.

```
varpy.is_equal(vp, x, y)
```

Check if literal **x** and literal **y** are the equal, that is bound to the
same variable or are bound to the same constant.

```
varpy.set_level(vp, l)
```

Set current level to **l**. Note that level 0 is treated
as constant level. So be careful when setting level to 0.

```
varpy.keep_level(vp, l)
```

Keep all bindings on level **l** by removing the undo information.

```
varpy.move_level(vp, src, dst)
```

Move bindings from level **src** to level **dst**.

```
varpy.undo_level(vp, l)
```

Undo all bindings on level **l**.

```
varpy.undo(vp)
```

Undo bindings typically after an nbind. Undo will undo all bindings
until a decision and flip the variable if not already flipped.

```
varpy.bcp(vp [,[x1,..,xn] [,all]])
```

Run value propagation. Return **True** if no
contradiction is found, **False** otherwise.

[EXPERIMENTAL]

If literals **x1**..**xn** are given they are checked for
"turbo" rule, that is, if all clauses that xi
is a part of are true regardless of the value of xi.
If 'all' is true then all xi's must be true for the
rule to hold. If turbo rule is successful then
varpy.**turbo** is returned.

[EXPERIMENTAL]

```
varpy.nbcp(vp)
```

decide and bind next unbound variables until either
no more variables to bind, return True,
or a contradicion is reached, then return False.
nbcp can be use with undo to implement a tight loop
for simple backtracking.

```
    def bt(vp):
        while not varpy.nbcp(vp):
          if varpy.undo(vp) == False:
            return False # contradiction
        return True # model
```

```
varpy.add_clause(vp, [x1,...,xn] [,clause_set])
```

Create a new clause, given as a literal list and return the
new clause index. All varables indices must already have been
created by calling add_variable. The clause create is installed
in one of four clause sets: 'delta', 'gamma', 'alpha', 'beta'.
The 'delta' clauseset is use to store the "problem" formula
clauses while 'gamma' is used for storing learnt clauses.
However the conflict clause(s) created by varpy.conflict are created in
'alpha' and may then, by user, moved into 'gamma'.

```
varpy.get_clause(vp, cix [, skip | varpy.undefined [, raw]] )
```

Retrive a clause as list given the clause index **cix**.
If literal **x** is given then literal **x** is removed
from the clause list returned. if **raw** is **True** then literals
bound on level=0 are also return as normal, otherwise they are
removed. If **False** or the clause is dead and empty list is
returned (fixme).

```
varpy.find_clause(vp, [x1,...,xn])
```

Check if the clause [**x1**,...,**xn**] exist among the clausesets.
return clause index if found, return **False** otherwise.

```
varpy.compress_clause(vp,  cix | [x1,...,xn])
```

Return a compressed version of the clause [**x1**,...,**xn**],
or the clause given by the clause index cix.
It writes a utf8 like code with 0x80 bit for continuation bit and
7-bits per byte for integer value. The LSB is coded as the literal sign.

-1000 is translated to unsigned by

```
2*1000 + 1 = 0xb11111010001
```

which is divided in groups of 7 bits starting from LSB

```
(1)1010001,(0)0001111
```

while 1000 is translated to

```
2*1000 = 0xb11111010000
```

which is divided in groups of 7 bits starting from LSB

```
(1)1010000,(0)0001111
```

Then sequence of coded literals are then terminated with a zero.

```
varpy.clause_info(vp, cix, item)
```

Get information about clause given by clause index cix

- 'length'
- 'jump'
- 'status'
- 'watch0'
- 'watch1'
- 'watch'

```
varpy.variable_info(vp, x, item)
```

Get information about variable x

- 'implication'
- 'implication_clause'
- 'implication_pos'
- 'level'

- 'phase'
- 'is_atom'
- 'degree'
- 'symbol'

```
varpy.literal_info(vp, x, item)
```

Get information about literal x

- 'degree'
- 'user'
- 'edge'
- 'symbol'

```
varpy.del_clause(vp, cix | [x1,...,xn])
```

Delete clause **cix** or [**x1**,...,**xn**] from clausesets.
NOTE: When deleting clauses by giving it as a list,
then hashing may be enable (varpy.config(vp, 'hash', True))
to gain reasonable speed.

```
varpy.clean_clause(vp, cix)
```

Cleanup clause by removing all false literals on level 0.
if clause is contradictory then exception is raised, else
varpy.**ok** is returned.

```
varpy.clean_edges(vp, x)
```

Remove **x** edges, that is clauses on form [-x,y] where y
is constant.

```
varpy.get_clauses(vp, cix, skip, raw)
```

Return a list of literals given by clause index **cix**.
Remove the literal **skip** from the returned list.
Also remove literals on level 0 if **raw** is **False**.

```
varpy.get_decision(vp, l)
```

Get literal on decision level **l**.

```
varpy.get_undo_state(vp, l)
```

DEBUG

Return the undo state on level **l**

- varpy.**set**
- varpy.**toggle**
- varpy.**done**
- varpy.**undef**

```
varpy.get_bindings(vp, level, clauseinfo, as_trail, as_tuple)
```

```
varpy.get_nbindings(vp, count clauseinfo, as_trail)
```

```
varpy.get_number_of_bindings(vp, l)
```

```
varpy.order_sort(vp, key1, key2, arg)
```

```
varpy.order_first(vp, [x1,...,xn])
```

```
varpy.order_last(vp, [x1,...,xn])
```

```
varpy.next_unbound(varp [, previous])
```

Return the next unbound literal in the current variable order.
If **previous** is given then start looking for unbound literals
from that point.

```
varpy.queue_first(varp)
```

DEBUG Return the first literal on the bcp queue.

```
varpy.queue_next(vp, x)
```

DEBUG Return the next literal on the bcp queue, following literal **x**,
that must previously being returned from varpy.queue_first of varpy.queue_next.

```
varpy.queue_clear(varp)
```

Remove all literals enqueued on the bcp queue by calls to
varpy.bind or varpy.decide.

```
varpy.add_symbol(vp, x, string|term)
```

Associate a term or string to to a variable **x**, for example the
name of the variable. The term or string must not be assoicated with
other variables or exception will occur.

```
varpy.find_symbol(vp, string|term)
```

Given a string or term return the variable assoicated.
If no assoication is found **False** is returned.

```
varpy.use_clause(vp, cix)
```

Mark clause **cix** as "used" by assigning a timestamp count to the
clause. This timestamp is based on the bcp counter, the number of
bcp that has been run since **vp** instance was created.

```
varpy.bump(vp, x, n)
```

"bump" a variable **x** to move it in the dynamic variable order.
Either bump value **n** is one of

- 'next', move variable to the top position, next variable to be assigned
- 'log2', bump value is calculated to log2('**number-of-variables**')
- 'log10', bump value is calculated to log10('**number-of-variables**')
- 'rank', bump value is set to the length of the implication clause.

Or the **n** is a floating point ration between zero and one that will
give the the number of steps to move, 0.1 means move 10% in number of
variables.
Or the value is an integer that gives an absolute number of steps to move.

```
varpy.subscribe(vp, flag|[flag])
```

Flags

- varpy.**variable**
- varpy.**atom**
- varpy.**number_of_variables**
- varpy.**number_of_bound_variables**
- varpy.**number_of_subst_variables**
- varpy.**number_of_clauses**
- varpy.**number_of_dead_clauses**
- varpy.**max_level**
- varpy.**max_bound**
- varpy.**min_level**

```
varpy.clauseset_size(vp, s)
```

Where **s** is one of 'delta', 'gamma', 'alpha', 'beta'

```
varpy.clauseset_offset(vp, s)
```

Get offset where **s** is one of 'delta', 'gamma', 'alpha', 'beta'

```
varpy.clauseset_offset(vp, s, offset)
```

Set offset where **s** is one of 'delta', 'gamma', 'alpha', 'beta'

```
varpy.clauseset_sort(vp, s)
```

Sort clauses in clause set **s** where **s** is one of
'delta', 'gamma', 'alpha', 'beta'

```
varpy.clauseset_first(vp, s)
```

get clause index of first clause in clauseset **s**

```
varpy.clauseset_next(vp, s)
```

get clause index to the next clause in clauseset **s**

```
varpy.set_user_count(vp, x, count)
```

Set user value for literal **x** to **count**.

```
varpy.conflict(vp, level, bump, i)
```

Do conflict analysis, called with level where the conflict i was found
and the **bump** factor that is applied to variables involved in the conflict.
Return value is a clause index in clauseset 'alpha'. This
clause may then be minimized and later moved to 'gamma'.

```
varpy.minimize(vp, cix)
```

Minimize clause, may be called after varpy.conflict and requires
that literals and levels are set like after the conflict.

```
varpy.move_clause(vp, cix, set)
```

Move clause **cix** to clauseset **set**.
NOTE that this function is currently limited to clause in 'alpha'
and set must be 'gamma'

```
varpy.unmark(vp)
```

Clear all marks

```
varpy.mark(vp, l | [x1,...,xn] | (x1,...,xn), [clear])
```

Mark variables **x1**..**xn** or all bindind on level **l**.
Optionally if **clear** is **False** then marks are concatinated to the
previous marks, otherwise the marks are cleared before adding new ones.

```
varpy.intersect_marks(vp, l | [x1,...,xn] | (x1,...,xn))
```

Keep all marks that are present in bindings on level **l** or
the literals **x1**...**xn**. Clear the other marks.

```
varpy.intersect_var(vp, x, l | [x1,...,xn]|(x1,...,xn), as_tuple)
```

Return all marks that are present in bindings on level **l** or
the literals **x1**..**xn**. Return the marks in a list if
_as_tuple if **False** or as a tuple if **as_tuple** is **True**.

```
varpy.get_marked(vp, as_tuple)
```

Return the marks in a list if
_as_tuple if **False** or as a tuple if **as_tuple** is **True**.